

Lecture 1: Introduction to Programming, Flowcharts, and Basic Algorithms

1. The Fundamentals of Programming

Before writing logic, it is critical to understand how computers interpret human instructions.

- **Programming Language:** A formal language comprising sets of strings that produce various kinds of machine output. It is the medium used to communicate instructions to a computer.
- **Compilation Workflow:** Computers exclusively understand binary code (0s and 1s). A compiler acts as a translator, processing the human-readable source code into executable machine code.
- **Execution Pipeline:** The standard pipeline follows this sequence: Source Code → Compiler → Binary/Machine Code → Computer Executes (.exe).

Below is the standard boilerplate syntax for a C++ program to execute the fundamental "Hello World" command:

C++

```
// C++ program to display "Hello World"
// Header file for input output functions
#include <iostream>
using namespace std;

// Main() function: where the execution of program begins
int main()
{
    // prints hello world
    cout << "Hello World";

    return 0;
}
```

Note: The above code snippet demonstrates basic C++ syntax.

2. Flowcharts & Pseudocode Blueprint

When solving programming problems, jumping straight into syntax can lead to logical errors. The optimal strategy is: Understand the Problem → Figure out an approach → Write rough Pseudocode/Flowchart → Write Code.

- **Pseudocode:** A highly simplified, language-agnostic, high-level representation of computer logic used during the program design phase.

- **Flowchart:** A diagrammatic representation of an algorithmic approach, drawing out all steps sequentially.

Core Flowchart Components:

- **Terminator (Oval):** Specifies the exact start and end points of a program.
- **Input/Output (Parallelogram):** Denotes taking variables from the user or displaying results.
- **Process (Rectangle):** Represents mathematical operations, assignments, or state changes (e.g., $i = i + 1$).
- **Decision Making (Diamond):** Represents conditional checks (e.g., `is a < b?`) that branch the flow into True/Yes or False/No paths.
- **Connector (Circle):** Connects different parts of a flowchart, heavily utilized when defining external functions.
- **Arrows:** Dictate the directional flow of code execution.

3. Core Operator: Modulo (%)

The modulo operator is foundational for parity checks and cyclic algorithms.

- **Definition:** The `%` operator returns the strictly mathematical remainder after dividing two integers a and b .
- **Parity Check:** If $N \% 2 == 0$, the number N is even. Otherwise, N is odd.
- **Edge Case Property:** If $a < b$, the modulo operation $a \% b$ simply evaluates to a .

Problem-Solving Deconstruction

Problem 1: Valid Triangle Check

1. Problem Statement & Constraints: Given three side lengths a , b , and c , determine if they can form a mathematically valid triangle.

2. Core Intuition: This problem relies on the Triangle Inequality Theorem. A triangle is valid if and only if the sum of any two sides is strictly greater than the third side. All three permutations of this condition must hold true simultaneously.

3. Algorithmic Steps:

As depicted in the reference file 01.jpg, the logic follows a sequential branching structure:

1. Initialize the program and read inputs a , b , and c .
2. Check the first condition: Is $a + b > c$? If False, immediately print "invalid" and terminate.

3. If True, check the second condition: Is $b + c > a$? If False, print "invalid" and terminate.
4. If True, check the final condition: Is $c + a > b$? If False, print "invalid" and terminate.
5. If all three conditions evaluate to True, print "valid" and terminate the program.

4. Dry Run:

Input: $a = 3, b = 4, c = 5$

1. Check $3 + 4 > 5 \rightarrow 7 > 5$ (True). Proceed.
2. Check $4 + 5 > 3 \rightarrow 9 > 3$ (True). Proceed.
3. Check $5 + 3 > 4 \rightarrow 8 > 4$ (True). Proceed.
4. Output: "valid".

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	Performs a constant number (maximum 3) of logical checks regardless of input size.
Space Complexity	$O(1)$	Allocates a fixed amount of memory for variables a , b , and c .

Problem 2: Print All Odd Numbers from 1 to N

1. Problem Statement & Constraints: Print all odd numbers starting from 1 up to a given boundary N (inclusive).

2. Core Intuition:

Instead of checking every number with a modulo operator ($i \% 2 \neq 0$), we can optimize the approach. We know the first odd number is 1, and consecutive odd numbers are separated by exactly 2. We can simply start at 1 and iteratively add 2 until we exceed N .

3. Algorithmic Steps:

Based on the loop execution shown in 02.jpg:

1. Read the target limit N .
2. Initialize an iterator variable `num` to 1.
3. Evaluate the loop condition: Is `num <= N`?
4. If the condition is False, exit the loop and terminate the program.
5. If the condition is True, print the current value of `num`.
6. Update the iterator by adding 2 (`num = num + 2`) and jump back to Step 3.

4. Dry Run:

Input: $N = 5$

1. `num = 1` . Check $1 \leq 5$ (True). Print `1` . Update `num = 3` .
2. `num = 3` . Check $3 \leq 5$ (True). Print `3` . Update `num = 5` .
3. `num = 5` . Check $5 \leq 5$ (True). Print `5` . Update `num = 7` .
4. `num = 7` . Check $7 \leq 5$ (False). Terminate.

Output: `1, 3, 5`

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The loop executes roughly $N/2$ times, which simplifies asymptotically to linear time.
Space Complexity	$O(1)$	Only one auxiliary variable (<code>num</code>) is tracked in memory.

Problem 3: Calculate Factorial of N ($N!$)

1. Problem Statement & Constraints: Calculate the factorial of a given integer N , mathematically defined as $N! = N \times (N - 1) \times (N - 2) \cdots \times 1$.

2. Core Intuition:

We must maintain a running product (accumulator). By looping a counter from 1 up to N , we continuously multiply our accumulator by the current loop counter.

3. Algorithmic Steps:

Following the structural flow illustrated in 03.jpg:

1. Read the input value N .
2. Initialize a loop counter `num = 1` and an accumulator `ans = 1` .
3. Evaluate the boundary check: Is `num <= n` ?
4. If False, the loop breaks. Print `ans` (the final computed factorial) and end.
5. If True, update the accumulator: `ans = ans * num` .
6. Increment the loop counter: `num = num + 1` and return to Step 3.

4. Dry Run:

Input: $N = 4$

1. `num=1` , `ans=1` . Check $1 \leq 4$ (True). `ans = 1 * 1 = 1` . `num = 2` .
2. `num=2` , `ans=1` . Check $2 \leq 4$ (True). `ans = 1 * 2 = 2` . `num = 3` .
3. `num=3` , `ans=2` . Check $3 \leq 4$ (True). `ans = 2 * 3 = 6` . `num = 4` .

4. `num=4` , `ans=6` . Check $4 \leq 4$ (True). `ans = 6 * 4 = 24` . `num = 5` .
5. `num=5` , `ans=24` . Check $5 \leq 4$ (False). Break.
6. Print `24` .

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The algorithm loops exactly N times to calculate the running product.
Space Complexity	$O(1)$	Constant memory allocation for the <code>ans</code> and <code>num</code> state variables.

Problem 4: Check if N is Prime

1. Problem Statement & Constraints: Given an integer N , determine if it is a prime number (a number divisible only by 1 and itself).

2. Core Intuition: To prove N is prime, we must verify that it lacks any divisors other than 1 and N . We can use a brute-force loop starting from a divisor of 2 up to $N-1$. If the modulo operation (`N % div`) evaluates to 0 at any point, a perfect divisor has been found, meaning the number is not prime.

3. Algorithmic Steps:

1. Read input N .
2. Initialize a divisor tracking variable `div = 2` .
3. Check loop constraint: Is `div < N` ?
4. If False (we checked all numbers up to $N-1$ without finding a factor), print "Yes" (N is prime) and terminate.
5. If True, check the modulo condition: Is `N % div == 0` ?
6. If the modulo is 0, print "No" (a factor exists) and immediately exit the program.
7. If the modulo is not 0, increment the divisor `div = div + 1` and jump back to Step 3.

4. Dry Run:

Input: $N = 5$

1. `div=2` . $2 < 5$ (True). Check `5%2 == 0` (False). `div = 3` .
2. `div=3` . $3 < 5$ (True). Check `5%3 == 0` (False). `div = 4` .
3. `div=4` . $4 < 5$ (True). Check `5%4 == 0` (False). `div = 5` .
4. `div=5` . $5 < 5$ (False). Loop ends.
5. Print "Yes". Number is prime.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	Brute force approach scales linearly. (Note: This can be heavily optimized to $O(\sqrt{N})$ by stopping the loop at $\sqrt{N}$).
Space Complexity	$O(1)$	No extra memory scaling with input size.